SCREENSHOT AND EVIDENCE CHECKLIST - WEEK 2 HANDS-ON TASKS

Status: all six hands-on tasks are backed by real evidence. Nothing in this folder is a mock-up, a reconstruction, or an instruction sheet waiting to be replaced.

Every image carries a badge in its top-right corner:

Badge	Meaning
`REAL OUTPUT`	Captured from a tool that actually ran - Wireshark 4.6.9, `tshark`, Snort 2.9.20, a genuine Ubuntu 22.04.5 VM, or this host's own `nslookup`/`ping`/`curl`/`ipconfig`
`REAL DATA`	A table or chart computed from a real artefact - the supplied capture, the Apache access log, or a real capture file

The `REAL OUTPUT` screenshots of Wireshark are pictures of the real Wireshark GUI with the capture loaded and the display filter applied. Each one was produced with one Wireshark launch per filter (`Wireshark.exe -r <capture> -Y "<filter>"`), and the launch script verified from the window title that the intended capture file had actually loaded before saving the image - so the packet list in a screenshot really is the result of the filter in its filename.

A. Wireshark - the supplied capture file

#	Evidence	How it was produced	File
A1	Capture loaded, 1171 packets, 36.880 s	Wireshark GUI, file open	`1a Wireshark capture loaded.png`
A2	All source and destination addresses	`Statistics > Endpoints`	`1s Wireshark Endpoints.png`
A3	Conversation packets and bytes	`Statistics > Conversations`	`1t Conversations.png`
A4	Protocol hierarchy - SMB2, DCERPC, Netlogon, DRSUAPI	`Statistics > Protocol Hierarchy`	`1r Protocol Hierarchy Wireshark.png`
A5	The HTTP-formatted SSDP requests	`udp.port == 1900`	`1k Wireshark SSDP M-SEARCH.png`
A6	Proof there is no DNS	`dns` - empty packet list	`1l Wireshark dns empty.png`
A7	The SMB session	`tcp.port == 445`	`1o Wireshark tcp port 445 SMB.png`
A8	Netlogon and DRSUAPI (the DCSync interface)	`dcerpc`	`1p Wireshark dcerpc AD replication.png`
A9	The OpenVPN tunnel	`openvpn`	`1q Wireshark openvpn.png`
A10	Capture facts, IPs, ports, flags, DNS and HTTP findings	Computed from the capture	`1b-1h`

B. Wireshark - traffic generated for this task

#	Evidence	Filter / command	File
B1	ICMP echo requests and replies	`icmp`	`2g Wireshark icmp filter.png`

#	Evidence	Filter / command	File
B2	DNS queries	`dns`	`2h Wireshark dns filter.png`
B3	Reverse (PTR) lookups	`dns.qry.type == 12`	`2i Wireshark dns PTR query.png`
B4	A real plain-HTTP request	`http`	`2j Wireshark http filter.png`
B5	HTTPS conversations	`tcp.port == 443`	`2k Wireshark tcp port 443 TLS.png`
B6	TLS ClientHello and the SNI field	`tls.handshake.type == 1`	`2l Wireshark TLS ClientHello.png`
B7	A complete DHCP DORA exchange	`dhcp`	`2m Wireshark dhcp DORA.png`
B8	My own DNS, ping, HTTP, download and DHCP output	`nslookup`, `ping`, `curl`, `ipconfig` on host `EDITH`	`2a-2f`
B9	The real capture files used and their contents	`tshark` on the Wireshark project's test captures	`2n-2r`

C. Linux logs - a real Ubuntu 22.04.5 VM

A genuine Ubuntu 22.04.5 LTS cloud image was booted as a QEMU guest (pure emulation, no administrator rights needed on the host). The user `analyst` was created, privileged commands were run through `sudo`, and the logs were read back. The VM's console output is preserved verbatim in `_build/artifacts/raw/linux_evidence.txt`.

#	Evidence	Command that produced it	File	
C1	My own sudo session in auth.log	`grep -i 'sudo' /var/log/auth.log \`	tail -20`	`3a auth.log sudo entries.png`
C2	The elevated commands (`COMMAND=` fields)	`grep 'COMMAND=' /var/log/auth.log`	`3e auth.log commands.png`	
C3	The same events in syslog	`grep -i 'sudo' /var/log/syslog \`	tail -10`	`3b syslog sudo entry.png`
C4	Hostname, release, kernel, disk, log counts	`id`, `uname -a`, `lsblk`, `df -h /`	`3c Ubuntu VM facts.png`	
C5	The log files and rsyslog state	`ls -l /var/log/auth.log /var/log/syslog ...`	`3d Linux log files.png`	
C6	The raw file format	`head -8 /var/log/auth.log`	`3f auth log format.png`	

D. Web server log analysis

#	Evidence	File
D1	Top IP addresses and the top 3	`4a Apache Top IP Addresses.png`, `4b Top 3 Source IP Addresses.png`
D2	Status code distribution	`4c Status Code Distribution.png`
D3	Scanner-style requests and the IPs behind them	`4d Suspicious Requests.png`

E. MITRE ATT&CK mapping

#	Evidence	File
E1	Behaviour to tactic to technique to sub-technique, with detection data sources	`5a ATTACK Mapping Table.png`
E2	The three behaviours chained into one intrusion	`5b ATTACK Tactic Chain.png`

F. Snort - installed and run for real

Snort 2.9.20-WIN64 (Build 82) was installed without administrator rights and run against five real capture files.

#	Evidence	Command that produced it	File
F1	Version and build	`snort.exe -V`	`6b Snort version.png`
F2	`Snort successfully validated the configuration!`	`snort.exe -T -c local.rules`	`6b Snort version.png`
F3	The local rules	`local.rules`	`6c Snort local rules.png`
F4	Real ICMP alerts - the rule firing	`snort.exe -A console -q -c local.rules -r icmp.pcapng`	`6a Snort local rules and real alerts.png`
F5	Alert counts across all five captures	Five `snort.exe -r` runs	`6d Snort alert summary.png`
F6	Verbatim alert text	Five `snort.exe -r` runs	`6e Snort alerts detail.png`
F7	66 SYN alerts on the supplied capture	`snort.exe -r wireshark_capture.bin`	`6f Snort console alerts supplied.png`

Reproducing any of it

Everything in this folder can be regenerated from source. The generator scripts live in `_build/` and are described in the top-level `README.md`:

```
_build/analyse_pcap.py      # the supplied capture, dissected in pure Python
_build/make_images.py       # the tables and charts in this folder
_build/collect_real.py      # runs tshark 4.6.9 and snort 2.9.20, captures their real output
_build/make_real_images.py  # renders that real output into the images above
_build/extract_linux.py     # the Ubuntu VM's auth.log / syslog evidence
_build/make_seed.py         # the cloud-init seed used to build and drive that VM
_build/tools/ws_*.ps1        # the Wireshark GUI screenshot automation
```

Housekeeping

- Every file name starts with its task number (`0`-`6`) so the folder sorts in task order.
- No screenshot is cropped at the filter bar - the filter is the proof of what is shown.
- Host names, user names and timestamps are left visible; they are part of the evidence.